Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра “Вычислительная техника”

Отчёт

по дополнительному заданию
по курсу “Трансляторы и Компиляторы”
на тему
“Оптимизация структуры посевных площадей Пензенской области
на 2026–2028 гг. методом линейного программирования
на основе прогноза урожайности”

Выполнил студенты гр. 24ВВП1:
Бауэр А.А.
Гусев Е.В.
Медведев Е.А.

Приняли:
д.т.н., профессор Дубинин В. Н.
к.т.н., доцент Карамышева Н. С.

Пенза 2026

1. Цель работы

По прогнозу урожайности оценить структуру посевных площадей Пензенской области на 2026 - 2028 годы и найти распределение гектаров с большей маржой.

2. Задание

Для зерна, сахарной свёклы, подсолнечника, картофеля и овощей открытого грунта распределить 1254,5 тыс. га так, чтобы маржинальный доход был максимальным. Прогноз урожайности используется готовым.

3. Ход выполнения работы

Программа написана на C++, в которой учли все собранные данные по заданию.

Маржа гектара считается как цена, умноженная на урожайность, минус затраты. 65 % затрат не зависят от собранного урожая и берутся по 2024 году, остальные растут вместе с урожаем. Сумма площадей каждый год равна фонду 2025 года. За один год культура сдвигается не сильнее заданного шага. Сбор свёклы ограничен 3200 тыс. т, площадь подсолнечника - 40 % зернового клина.

В базовом плане к 2028 году зерно занимает 821,6 тыс. га, свёкла 69,8, подсолнечник 328,6, картофель 28,0, овощи 6,5. За три года этот план даёт на 5017 млн руб. маржи больше, чем посевы 2025 года.

4. Анализ полученных данных

Таблица 1 - Цены, себестоимость и площади 2025 года

Культура	Цена, руб./т	Себестоимость, руб./т	Площадь 2025, тыс. га
Зерновые и зернобобовые	12 727	9 350	852,0
Сахарная свёкла	3 757	2 980	65,0
Подсолнечник	33 989	20 490	313,0
Картофель	24 098	14 090	20,0
Овощи открытого грунта	20 130	14 880	4,5

4.1. Модель

Маржа одного гектара равна цене, умноженной на урожайность, за вычетом затрат. Затраты делятся на две части. 65 процентов не зависят от собранного урожая: это обработка почвы, семена и защита растений, они считаются по гектару 2024 года. Остальные 35 процентов растут вместе с урожаем: уборка, подработка и вывоз. Для зерновых при медиане 2026 года это даёт затраты 29 688 руб./га, выручку 45 690 руб./га и маржу 16 001 руб./га.

На каждый год наложены такие условия. Сумма площадей равна 1254,5 тыс. га. За один год культура не может сдвинуться сильнее заданного шага. У каждой культуры есть постоянный пол и потолок. Сбор сахарной свёклы не выше 3 200 тыс. т: это ориентир мощности заводов, чуть выше фактического сбора 2025 года (2 989 тыс. т). Площадь подсолнечника не больше 40 процентов площади зерновых. В 2025 году это соотношение составляло 36,7 процента. Задача решается двухфазным симплекс-методом. Следующий год отталкивается от плана предыдущего, а не заново от факта 2025 года.

4.2. Результаты расчёта

При медианной урожайности 2026 года гектар культур даёт разную маржу. Именно эта величина решает, куда модель отдаёт свободную землю, пока не упрётся в ограничение.

Таблица 2 - Маржа одного гектара при медиане прогноза на 2026 год

Культура	Урожайность, т/га	Затраты, руб./га	Выручка, руб./га	Маржа, руб./га
Зерновые и зернобобовые	3,59	29 688	45 690	16 001
Сахарная свёкла	44,61	113 356	167 600	54 243
Подсолнечник	2,06	41 414	70 017	28 603
Картофель	14,74	190 195	355 205	165 010
Овощи открытого грунта	24,94	362 960	502 042	139 083

В базовом сценарии картофель и овощи наращиваются до принятых потолков, свёкла - до загрузки заводов, подсолнечник - до севооборотной доли. Зерно отдаёт высвобождаемые гектары. Рентабельность затрат базового плана составляет 59,2 процента в 2026 году, 61,5 в 2027 году и 64,6 в 2028 году.

Таблица 3 - Площади базового сценария, тыс. га

Культура	2025	2026	2027	2028	К 2028
Зерновые и зернобобовые	852,0	823,0	821,2	821,6	-30,4
Сахарная свёкла	65,0	71,7	70,3	69,8	+4,8
Подсолнечник	313,0	329,2	328,5	328,6	+15,6
Картофель	20,0	25,0	28,0	28,0	+8,0
Овощи открытого грунта	4,5	5,6	6,5	6,5	+2,0
Итого	**1254,5**	**1254,5**	**1254,5**	**1254,5**	**0**

Таблица 4 - Маржа базового плана и выигрыш против площадей 2025 года

Показатель	2026	2027	2028
Маржа плана, млн руб.	31 383	33 264	35 359
Маржа при площадях 2025 года, млн руб.	30 038	31 421	33 530
Выигрыш перестройки, млн руб.	1 345	1 843	1 829

Суммарный выигрыш базового сценария за три года составляет 5 017 млн руб., или 5,02 млрд руб. Это маржа, а не чистая прибыль и не эффект за вычетом строительства. По остальным сценариям суммарный выигрыш против структуры 2025 года такой: верх интервала урожайности - 4,66 млрд руб., низ интервала - 4,84 млрд руб., линейный тренд 2011-2025 годов - 4,59 млрд руб., снижение цен картофеля и овощей на 30 процентов - 2,20 млрд руб.

Таблица 5 - Прирост маржи в 2028 году при ослаблении одного ограничения

Что меняется	Прирост маржи, млн руб. в год
Картофель, потолок площади +1 тыс. га	147,7
Овощи, потолок площади +0,5 тыс. га	59,0
Сахарные заводы, мощность +100 тыс. т	76,9
Доля подсолнечника к зерну с 0,40 до 0,45	138,6

5. Результаты работы

Окно программы и файлы с числами показаны на рисунках 1–3.

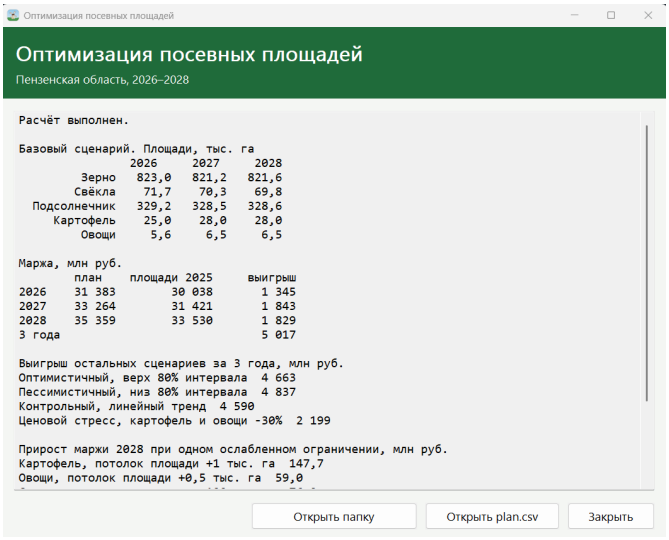


Рисунок 1 - Окно программы

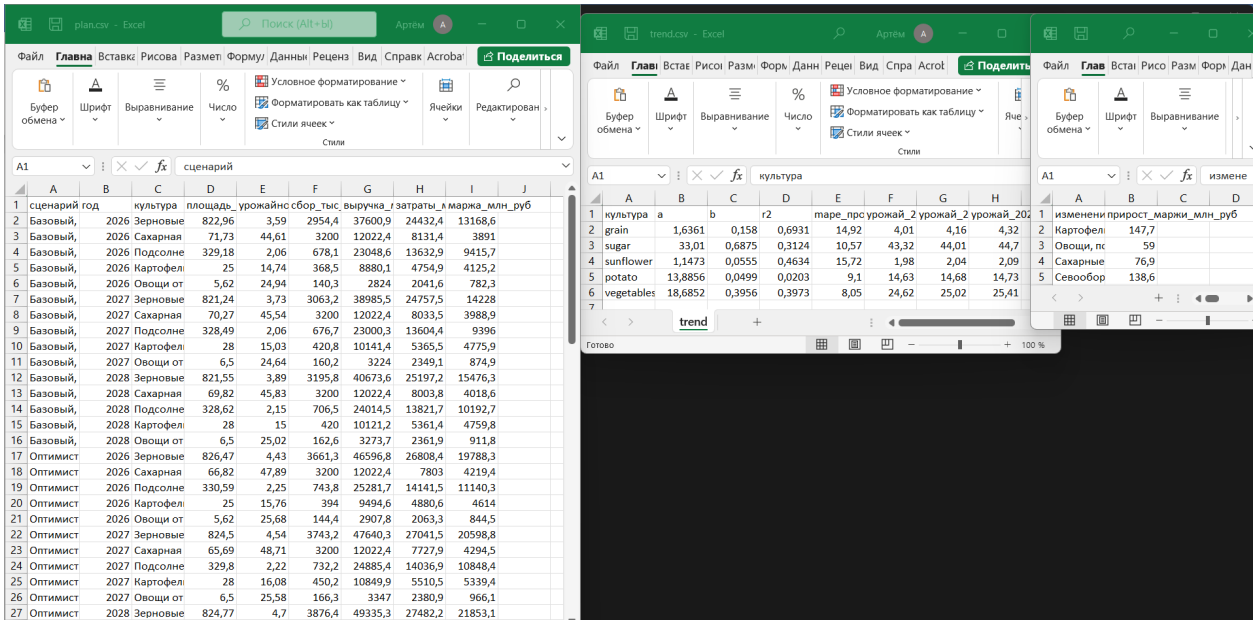


Рисунок 2 - Таблицы plan.csv, trend.csv и effects.csv

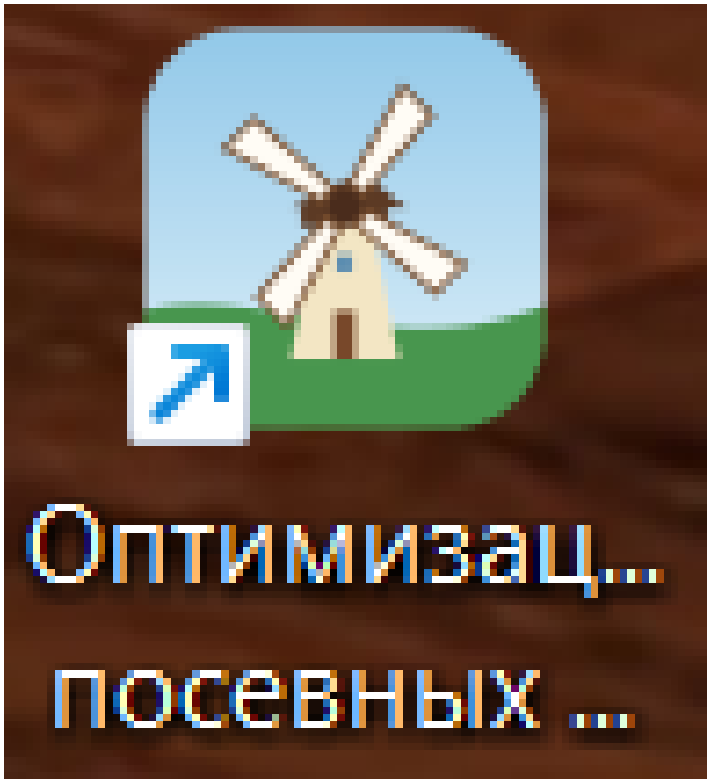


Рисунок 3 - Ярлык установленной программы

6. Выводы

При ценах 2025 года и медианном прогнозе урожайности свободный гектар выгодно отдавать в первую очередь картофелю и овощам, затем сахарной свёкле и подсолнечнику. Зерновые остаются самым большим клином, но часть их площади уходит под более доходные культуры. К 2028 году зерно занимает 821,6 тыс. га, что на 30,4 тыс. га меньше факта 2025 года, сахарная свёкла - 69,8 тыс. га (+4,8), подсолнечник - 328,6 (+15,6), картофель - 28,0 (+8,0), овощи открытого грунта - 6,5 (+2,0). Если сохранить структуру посевов 2025 года, за три года будет недополучено 5,02 млрд руб. маржинального дохода.

Сдвиг площадей останавливают не недостаток земли, а принятые ограничения. Сахарная свёкла в базовом сценарии выходит на мощность переработки 3 200 тыс. т, поэтому при росте урожайности под неё требуется меньше гектаров. Подсолнечник останавливает севооборот: его площадь не превышает 40 процентов зернового клина. Картофель и овощи к 2027 году

достигают потолков 28 и 6,5 тыс. га, которые отражают нехватку людей и хранения, а не пашни. Нижняя граница по зерну в 760 тыс. га не достигается.

Риск по урожаю при тех же ценах остаётся большим. Маржа оптимального плана 2028 года между верхней и нижней границей 80-процентного интервала различается на 17,18 млрд руб. Меняется и набор площадей: в пессимистичном сценарии заводы не заполняются, и свёкла упирается уже в годовой шаг и потолок 80 тыс. га. Отдельно проверена цена. Если картофель и овощи подешевеют на 30 процентов при базовой урожайности, овощи становятся слабо убыточными и опускаются на минимум 4 тыс. га, а картофель свой потолок не отдаёт, потому что маржа его гектара всё ещё выше зерновой. Наращивать овощи поэтому стоит только при договоре на сбыт.

Ослабление одного ограничения в 2028 году приносит за год 147,7 млн руб. маржи на каждую дополнительную тысячу гектаров картофеля, 59,0 млн руб. на дополнительные 0,5 тыс. га овощей, 76,9 млн руб. на дополнительные 100 тыс. т мощности сахарных заводов и 138,6 млн руб., если допустить долю подсолнечника 0,45 вместо 0,40. Это прирост на стороне урожая. Сметы хранилищ, техники и реконструкции заводов в материалах задания нет, поэтому по одной этой цифре нельзя решить, окупится ли сам инструмент.

План получен при закреплённых ценах 2025 года и себестоимости 2024 года. Доля постоянных затрат 0,65, мощность заводов 3 200 тыс. т и севооборотная норма 0,40 заданы явно. Районное деление, субсидии и погода конкретного года в модель не входили. Контрольный линейный тренд даёт близкую структуру и 4,59 млрд руб. суммарного выигрыша, так что вывод не держится на одной только таблице 10. У картофеля прямая тренда почти ничего не объясняет, коэффициент детерминации равен 0,02, поэтому в основной сценарий взята медиана исследования.

Ссылка на GitHub: https://github.com/aaaartemkaa/T-C_8LR

Листинг программы

Файл simplex.h

```
#pragma once
#include <string>
#include <vector>
// Двухфазный симплекс: максимум c*x при x >= 0
// Ограничение записывается как sum(a[j] * x[j]) ? b
// Фаза 1 выгоняет искусственные переменные и проверяет совместность.
// Фаза 2 максимизирует исходную цель. Столбец для ввода берётся по Бланду —
// первый с положительной оценкой, ЧТОБЫ ВЫРОЖДАЮЩИЙ БАЗИС НЕ ЗАЦИКЛИЛСЯ

enum class LpType {
    LE,// <=
    GE,// >=
    EQ // =
};

struct LpConstraint {
    std::vector<double> a;
    double b = 0;
    LpType type = LpType::LE;
};

enum class SolveStatus {
    Optimal,// найден максимум
    Infeasible,// система ограничений пуста
    Unbounded,// цель можно растить бесконечно
    Error// размерности не сошлись или кончился лимит итераций
};

struct SolveResult {
    SolveStatus status = SolveStatus::Error;
    std::vector<double> x;// только исходные переменные, ДОП. СТОБЦЫ СЮДА НЕ ПОПА-
ДАЮТ
    double objective = 0;// c*x. в задаче площадей это млн руб. маржи
    int iterations = 0;
    std::string message;
};

SolveResult maximize(const std::vector<double>& c, const std::vector<LpCon-
straint>& constraints);
     bool simplex_self_test(std::string& message);
```

Файл report.h

```
#pragma once
#include "forecast.h"
#include "model.h"
#include <filesystem>
#include <vector>
// Пишет: план площадей, коэффициенты тренда и прирост маржи
void write_results(const std::filesystem::path& directory,
    const TrendLine trends[N_CROPS],
    const std::vector<Trajectory>& trajectories,
          const std::vector<MarginalEffect>& marginals);
```

Файл model.h

```
#pragma once
#include "common.h"
#include <string>
#include <vector>

//Чем именно упёрлась площадь: в постоянный потолок, в годовой шаг или в завод
enum class LimitKind { Agronomic, Inertia, Capacity };

//Деньги одной культуры при заданных урожайности и площади.
struct CropMoney {
    double cost_ha = 0;
    double margin_ha = 0;
    double harvest_thous_t = 0;
    double revenue_mln = 0;
    double cost_mln = 0;
    double margin_mln = 0;
};

//Решение одного года. ref — площади, от которых считался годовой шаг
struct SolvedYear {
    int year = 0;
    double yield[N_CROPS]{};
    double area[N_CROPS]{};
    double ref[N_CROPS]{};
    double lo[N_CROPS]{};
    double hi[N_CROPS]{};
    LimitKind lo_kind[N_CROPS]{};
    LimitKind hi_kind[N_CROPS]{};
    bool ratio_active = false;// Подсолнечник вышел РОВНО НА ОДНУ ДОЛЮ ОТ ЗЕРНА
    bool ok = false;
    std::string error;
};

//Три года одного сценария. crops и params хранятся вместе с планом:
//у ценового стресса цены другие, и маржу нельзя считать по исходным культурам
struct Trajectory {
    std::string key;
    std::string title;
    std::string note;
    std::vector<Crop> crops;
    Params params;
    SolvedYear years[N_YEARS]{};
    bool ok = false;
    std::string error;
};

//Прирост маржи, если ослабить одно ограничение и пересчитать тот же год
//extra_margin_mln — разница с исходным планом, ещё без стоимости самого инстру-
мента
struct MarginalEffect {
    std::string title;
    std::string meaning;
    double extra_margin_mln = 0;
    bool computed = false;
    std::string error;
};

//Затраты и маржа гектара, руб./га. Постоянная часть берётся с урожайности 2024
года,
//Переменная — с урожайности того года, который сейчас считается
double cost_per_ha(const Crop& crop, const Params& params, double yield_t_ha);
double margin_per_ha(const Crop& crop, const Params& params, double yield_t_ha);
```

```
CropMoney money_of(const Crop& crop, const Params& params, double yield_t_ha, dou-
ble area_thous_ha);
double total_margin(const std::vector<Crop>& crops, const Params& params, const
double yield[N_CROPS], const double area[N_CROPS]);

//Один год: пересечь шаги и агрономические границы, добавить мощность завода и
долю подсолнечника, решить симплекс
//reference — база годового шага
SolvedYear optimize_year(const std::vector<Crop>& crops, const Params& params, int
year, const double yield[N_CROPS], const double reference[N_CROPS], double to-
tal_area);

//2026 считается от факта 2025 года, 2027 — от плана 2026-го, 2028 — от плана
2027-го
Trajectory run_trajectory(const std::string& key, const std::string& title,
    const std::vector<Crop>& crops, const Params& params, double total_area,
    const double yields[N_YEARS][N_CROPS]);

//Четыре отдельных пересчёта года year: +1 тыс. га картофеля, +0,5 тыс. га овощей
std::vector<MarginalEffect> marginal_effects(const std::vector<Crop>& crops, const
Params& params,
         const SolvedYear& year, double total_area);
```

Файл forecast.h

```
#pragma once
#include "common.h"
#include <vector>
// Прямая МНК:
// урожайность = intercept + slope * (год - origin)
// origin — первый год окна (у нас 2011), поэтому intercept равен уровню на 2011 год
// R2 показывает, какая доля разброса легла на прямую. MAPE — средняя относительная ошибка на тех же годах
struct TrendLine {
    double intercept = 0;
    double slope = 0;
    double r2 = 0;
    double mape = 0;
    int n = 0;
    int origin = 2011;
};

// Окно включительно. Годы вне [year_from, year_to] в оценку не входят:
// ряд до 2011 года другой по технологиям, мешать его с современным нельзя
TrendLine fit_trend(const std::vector<YearRow>& history, int crop, int year_from, int year_to);
     double trend_at(const TrendLine& line, int year);
```

Файл data.h

```
#pragma once
#include "common.h"
#include <filesystem>
#include <stdexcept>
#include <string>
#include <vector>
// Всё, что программа читает с диска, уже разложенное по культурам
//total_area — сумма площадей 2025 года, земельный фонд пяти культур. Он не меня-
ется по годам
struct InputData {
    std::vector<Crop> crops;
    Params params;
    std::vector<YearRow> history;
    Band bootstrap[N_CROPS][N_YEARS]{};
    double total_area = 0;
};

// Ищет yield_area.csv, economics.csv и forecast_bootstrap.csv в каждой папке из
roots  и в её подпапке data
     InputData load_input(const std::vector<std::filesystem::path>& roots);
```

Файл common.h

```
#pragma once
#include <string>
// Индекс культуры один и тот же во всех массивах:
// 0 — зерно, 1 — свёкла, 2 — подсолнечник, 3 — картофель, 4 — овощи.
// Площади хранятся в тыс. га, урожайность в т/га. Деньги подписаны в полях ниже.

constexpr int N_CROPS = 5;
constexpr int N_YEARS = 3;
constexpr int PLAN_YEARS[N_YEARS] = {2026, 2027, 2028};

enum CropId {
    GRAIN = 0,
    SUGAR = 1,
    SUNFLOWER = 2,
    POTATO = 3,
    VEGETABLES = 4
};

//Одна культура и все коэффициенты, которые по ней читаются из economics.csv
struct Crop {
    std::string id;
    std::string name;
    double price_rub_t = 0;// цена реализации, руб./т
    double cost_rub_t = 0;// себестоимость 2024 года, руб./т
    double yield_base = 0;// урожайность того же 2024 года, т/га: на ней сидит
расчёт затрат
    double area_2025 = 0;// фактическая площадь, тыс. га; от неё стартует план
    double area_min = 0;// агрономический пол, тыс. га
    double area_max = 0;// агрономический потолок, тыс. га
    double inertia = 0;// 0.08 - за один год площадь можно сдвинуть не больше чем
на 8%
};

struct Params {
    // Доля затрат гектара, не зависящая от собранного урожая (обработка, семена,
защита)
    double fixed_cost_share = 0.65;
    // Сколько свёклы способны принять заводы, тыс. т
    double beet_capacity_thous_t = 3200;
    // x_подсолнечника <= sunflower_per_grain * x_зерна
    // ИНАЧЕ МОДЕЛЬ ВЫТЕСНИТ ЗЕРНО, без которого подсолнечник нечем чередовать
    double sunflower_per_grain = 0.40;
};

//Один год исторического ряда. Индекс массива - CropId
struct YearRow {
    int year = 0;
    double yield[N_CROPS]{};
    double area[N_CROPS]{};
};

//Прогноз на один год из таблицы 10: низ и верх 80% интервала и медиана, т/га
struct Band {
    double low = 0;
    double median = 0;
    double high = 0;
};

inline const char* crop_id(int index) {
    static const char* ids[N_CROPS] = {
        "grain", "sugar", "sunflower", "potato", "vegetables"
    };
    return ids[index];
```

```
}

//Ключ из CSV -> CropId. Строки файла могут идти в любом порядке
//Неизвестный ключ даёт -1, загрузчик на этом останавливается
inline int crop_index(const std::string& id) {
    for (int i = 0; i < N_CROPS; ++i) {
        if (id == crop_id(i)) {
            return i;
        }
    }
    return -1;
     }
```

Файл simplex.cpp

```
#include "simplex.h"
#include <cmath>
#include <limits>
#include <sstream>
#include <string>
#include <vector>
namespace {
    constexpr double EPS = 1e-8;
    struct Row {
        std::vector<double> a;
        double b = 0;
        int type = -1;
    };
    std::string fmt_vec(const std::vector<double>& x) {
        std::ostringstream out;
        out.setf(std::ios::fixed);
        out.precision(6);
        out << "[";
        for (size_t i = 0; i < x.size(); ++i) {
            if (i) {
                out << " ";
            }
            out << x[i];
        }
        out << "]";
        return out.str();
    }
}

SolveResult maximize(const std::vector<double>& c, const std::vector<LpCon-
straint>& constraints) {
    SolveResult result;
    const int n = static_cast<int>(c.size());
    const int m = static_cast<int>(constraints.size());
    if (n <= 0 || m <= 0) {
        result.message = "Пустая задача линейного программирования";
        return result;
    }

    std::vector<Row> rows(static_cast<size_t>(m));
    for (int i = 0; i < m; ++i) {
        if (static_cast<int>(constraints[static_cast<size_t>(i)].a.size()) != n) {
            result.message = "Длина строки ограничений не совпала с числом пере-
менных";
            return result;
        }
        rows[static_cast<size_t>(i)].a = constraints[static_cast<size_t>(i)].a;
        rows[static_cast<size_t>(i)].b = constraints[static_cast<size_t>(i)].b;
        if (constraints[static_cast<size_t>(i)].type == LpType::LE) {
            rows[static_cast<size_t>(i)].type = -1;
        } else if (constraints[static_cast<size_t>(i)].type == LpType::EQ) {
            rows[static_cast<size_t>(i)].type = 0;
        } else {
            rows[static_cast<size_t>(i)].type = 1;
        }
        // В таблице правая часть обязана быть >= 0.
        // Отрицательную строку умножаем на -1 и переворачиваем знак неравенства
        if (rows[static_cast<size_t>(i)].b < -EPS) {
            for (double& value : rows[static_cast<size_t>(i)].a) {
                value = -value;
            }
            rows[static_cast<size_t>(i)].b = -rows[static_cast<size_t>(i)].b;
```

```
            rows[static_cast<size_t>(i)].type = -
rows[static_cast<size_t>(i)].type;
        }
    }
    int n_slack = 0;
    int n_surplus = 0;
    int n_art = 0;
    for (const Row& row : rows) {
        if (row.type == -1) {
            ++n_slack;
        } else if (row.type == 1) {
            ++n_surplus;
            ++n_art;
        } else {
            ++n_art;
        }
    }

    // Столбцы таблицы: исходные x, слаки (<=), избытки (>=), искусственные, пра-
вая часть
    // <= сразу имеет базисный слак. >= и = стартуют с искусственной переменной,
    // иначе допустимого базиса нет и фаза 2 не с чего начинать
    const int n_col = n + n_slack + n_surplus + n_art;
    std::vector<std::vector<double>> table(static_cast<size_t>(m + 1), std::vec-
tor<double>(static_cast<size_t>(n_col + 1), 0.0));
    std::vector<int> basis(static_cast<size_t>(m), -1);
    const int slack_at = n;
    const int surplus_at = n + n_slack;
    const int art_at = n + n_slack + n_surplus;
    int slack_i = 0;
    int surplus_i = 0;
    int art_i = 0;

    for (int i = 0; i < m; ++i) {
        const Row& row = rows[static_cast<size_t>(i)];
        for (int j = 0; j < n; ++j) {
            table[static_cast<size_t>(i + 1)][static_cast<size_t>(j)] =
row.a[static_cast<size_t>(j)];
        }
        table[static_cast<size_t>(i + 1)][static_cast<size_t>(n_col)] = row.b;
        if (row.type == -1) {
            const int column = slack_at + slack_i++;
            table[static_cast<size_t>(i + 1)][static_cast<size_t>(column)] = 1.0;
            basis[static_cast<size_t>(i)] = column;
        } else if (row.type == 1) {
            const int surplus = surplus_at + surplus_i++;
            const int artificial = art_at + art_i++;
            table[static_cast<size_t>(i + 1)][static_cast<size_t>(surplus)] = -
1.0;
            table[static_cast<size_t>(i + 1)][static_cast<size_t>(artificial)] =
1.0;
            basis[static_cast<size_t>(i)] = artificial;
        } else {
            const int artificial = art_at + art_i++;
            table[static_cast<size_t>(i + 1)][static_cast<size_t>(artificial)] =
1.0;
            basis[static_cast<size_t>(i)] = artificial;
        }
    }

    auto pivot = [&](int pivot_row, int pivot_col) {
        const double pivot_value = ta-
ble[static_cast<size_t>(pivot_row)][static_cast<size_t>(pivot_col)];
        for (int j = 0; j <= n_col; ++j) {
            table[static_cast<size_t>(pivot_row)][static_cast<size_t>(j)] /=
pivot_value;
        }
```

```
        for (int i = 0; i <= m; ++i) {
            if (i == pivot_row) {
                continue;
            }
            const double factor = ta-
ble[static_cast<size_t>(i)][static_cast<size_t>(pivot_col)];
            if (std::fabs(factor) <= EPS) {
                continue;
            }
            for (int j = 0; j <= n_col; ++j) {
                table[static_cast<size_t>(i)][static_cast<size_t>(j)] -=
                    factor * ta-
ble[static_cast<size_t>(pivot_row)][static_cast<size_t>(j)];
            }
        }
        basis[static_cast<size_t>(pivot_row - 1)] = pivot_col;
        // Пыль после вычитания строк, иначе копит знак и ломает выбор опорного
элемента
        for (int i = 0; i <= m; ++i) {
            for (int j = 0; j <= n_col; ++j) {
                if (std::fabs(ta-
ble[static_cast<size_t>(i)][static_cast<size_t>(j)]) < 1e-12) {
                    table[static_cast<size_t>(i)][static_cast<size_t>(j)] = 0.0;
                }
            }
        }
    };

    auto run_phase = [&](bool phase1) -> int {
        for (int guard = 0; guard < 10000; ++guard) {
            ++result.iterations;
            std::vector<char> is_basic(static_cast<size_t>(n_col), 0);
            for (int basic_col : basis) {
                if (basic_col >= 0 && basic_col < n_col) {
                    is_basic[static_cast<size_t>(basic_col)] = 1;
                }
            }
            int enter = -1;
            for (int j = 0; j < n_col; ++j) {
                if (is_basic[static_cast<size_t>(j)]) {
                    continue;
                }
                if (!phase1 && j >= art_at) {
                    continue;
                }
                if (table[0][static_cast<size_t>(j)] > EPS) {
                    enter = j;// правило Бланда: не самый большой коэффициент, а
первый подходящий
                    break;
                }
            }
            if (enter < 0) {
                return 0;
            }
            int leave = -1;
            double best_ratio = std::numeric_limits<double>::infinity();
            int best_basis = 0;
            for (int i = 1; i <= m; ++i) {
                const double column = ta-
ble[static_cast<size_t>(i)][static_cast<size_t>(enter)];
                if (column <= EPS) {
                    continue;
                }
                const double ratio = ta-
ble[static_cast<size_t>(i)][static_cast<size_t>(n_col)] / column;
                const int basic_col = basis[static_cast<size_t>(i - 1)];
                if (ratio < best_ratio - 1e-12 ||
```

```
                    (std::fabs(ratio - best_ratio) <= 1e-12 && (leave < 0 ||
basic_col < best_basis))) {
                    best_ratio = ratio;
                    leave = i;
                    best_basis = basic_col;
                }
            }
            if (leave < 0) {
                return 1;
            }
            pivot(leave, enter);
        }
        return 2;
    };
    if (n_art > 0) {
        // Фаза 1 максимизирует минус сумму искусственных. Пока искусственная в
базисе,
        // её строку прибавляем к оценкам, чтобы у базисного столбца оценка стала
нулевой
        for (int j = art_at; j < art_at + n_art; ++j) {
            table[0][static_cast<size_t>(j)] = -1.0;
        }
        for (int i = 0; i < m; ++i) {
            if (basis[static_cast<size_t>(i)] >= art_at) {
                for (int j = 0; j <= n_col; ++j) {
                    table[0][static_cast<size_t>(j)] += ta-
ble[static_cast<size_t>(i + 1)][static_cast<size_t>(j)];
                }
            }
        }
        const int phase1 = run_phase(true);
        if (phase1 == 1) {
            result.status = SolveStatus::Unbounded;
            result.message = "Фаза 1 не ограничена";
            return result;
        }
        if (phase1 == 2) {
            result.status = SolveStatus::Error;
            result.message = "Фаза 1: превышен лимит итераций";
            return result;
        }
        double artificial_left = 0;
        for (int i = 0; i < m; ++i) {
            if (basis[static_cast<size_t>(i)] >= art_at) {
                artificial_left += table[static_cast<size_t>(i +
1)][static_cast<size_t>(n_col)];
            }
        }
        // Ненулевые искусственные в базисе значат, что исходные ограничения
несовместны.
        if (artificial_left > 1e-6) {
            result.status = SolveStatus::Infeasible;
            result.message = "Ограничения несовместны";
            return result;
        }
        for (double& value : table[0]) {
            value = 0;
        }
    }

    // Фаза 2. В строке оценок сначала стоит сама цель. Если x[j] уже базисная,
    // её оценку обнуляем, иначе симплекс попытается ввести переменную, которая
уже в базисе
    for (int j = 0; j < n; ++j) {
        table[0][static_cast<size_t>(j)] = c[static_cast<size_t>(j)];
    }
    for (int i = 0; i < m; ++i) {
```

```
        const int basic_col = basis[static_cast<size_t>(i)];
        if (basic_col < 0 || basic_col >= n) {
            continue;
        }
        const double coef = table[0][static_cast<size_t>(basic_col)];
        if (std::fabs(coef) <= EPS) {
            continue;
        }
        for (int j = 0; j <= n_col; ++j) {
            table[0][static_cast<size_t>(j)] -= coef * table[static_cast<size_t>(i
+ 1)][static_cast<size_t>(j)];
        }
    }

    const int phase2 = run_phase(false);
    if (phase2 == 1) {
        result.status = SolveStatus::Unbounded;
        result.message = "Целевая функция не ограничена сверху";
        return result;
    }
    if (phase2 == 2) {
        result.status = SolveStatus::Error;
        result.message = "Фаза 2: превышен лимит итераций";
        return result;
    }
    std::vector<char> is_basic(static_cast<size_t>(n_col), 0);
    for (int basic_col : basis) {
        if (basic_col >= 0 && basic_col < n_col) {
            is_basic[static_cast<size_t>(basic_col)] = 1;
        }
    }
    for (int j = 0; j < n_col; ++j) {
        if (j >= art_at || is_basic[static_cast<size_t>(j)]) {
            continue;
        }
        if (table[0][static_cast<size_t>(j)] > 1e-5) {
            result.status = SolveStatus::Error;
            result.message = "Симплекс остановился раньше оптимума";
            return result;
        }
    }

    result.x.assign(static_cast<size_t>(n), 0.0);
    for (int i = 0; i < m; ++i) {
        const int basic_col = basis[static_cast<size_t>(i)];
        if (basic_col >= 0 && basic_col < n) {
            double value = table[static_cast<size_t>(i +
1)][static_cast<size_t>(n_col)];
            if (std::fabs(value) < 1e-9) {
                value = 0;
            }
            result.x[static_cast<size_t>(basic_col)] = value;
        }
    }
    // Строка оценок хранит минус достигнутой цели: каждый шаг вычитает прирост из
правой части
    result.objective = -table[0][static_cast<size_t>(n_col)];
    result.status = SolveStatus::Optimal;
    return result;
}

bool simplex_self_test(std::string& message) {
    std::ostringstream errors;
    int failed = 0;
    auto expect = [&](const char* name, const SolveResult& solved, SolveStatus
status,
                      const std::vector<double>& x, double objective) {
```

```
        bool ok = solved.status == status;
        if (ok && status == SolveStatus::Optimal) {
            if (solved.x.size() != x.size() || std::fabs(solved.objective - objec-
tive) > 1e-4) {
                ok = false;
            }
            for (size_t i = 0; ok && i < x.size(); ++i) {
                if (std::fabs(solved.x[i] - x[i]) > 1e-4) {
                    ok = false;
                }
            }
        }
        if (!ok) {
            ++failed;
            errors << name << ": status=" << static_cast<int>(solved.status)
                << " obj=" << solved.objective << " x=" << fmt_vec(solved.x)
                << " (" << solved.message << ")\n";
        }
    };

    {
        std::vector<LpConstraint> cons(3);
        cons[0] = {{1, 0}, 4, LpType::LE};
        cons[1] = {{0, 1}, 6, LpType::LE};
        cons[2] = {{3, 2}, 18, LpType::LE};
        expect("границы", maximize({3, 5}, cons), SolveStatus::Optimal, {2, 6},
36);
    }
    {
        std::vector<LpConstraint> cons(4);
        cons[0] = {{1, 1}, 10, LpType::LE};
        cons[1] = {{1, 0}, 8, LpType::LE};
        cons[2] = {{1, 0}, 4, LpType::GE};
        cons[3] = {{0, 1}, 2, LpType::GE};
        expect("нижние границы", maximize({2, 3}, cons), SolveStatus::Optimal, {4,
6}, 26);
    }
    {
        std::vector<LpConstraint> cons(2);
        cons[0] = {{1, 1}, 10, LpType::EQ};
        cons[1] = {{-0.4, 1}, 0, LpType::LE};
        const double sun = 10.0 * 0.4 / 1.4;
        const double grain = 10.0 - sun;
        expect("равенство и пропорция", maximize({1, 10}, cons), SolveStatus::Op-
timal,
               {grain, sun}, grain + 10.0 * sun);
    }
    {
        std::vector<LpConstraint> cons(2);
        cons[0] = {{1}, 5, LpType::GE};
        cons[1] = {{1}, 2, LpType::LE};
        expect("несовместна", maximize({1}, cons), SolveStatus::Infeasible, {},
0);
    }
    {
        std::vector<LpConstraint> cons(1);
        cons[0] = {{1}, 1, LpType::GE};
        expect("не ограничена", maximize({1}, cons), SolveStatus::Unbounded, {},
0);
    }

    if (failed) {
        message = errors.str();
        return false;
    }
    message = "пройдена";
    return true;
```

```
}
```

Файл report.cpp

```
#include "report.h"
#include <fstream>
#include <sstream>
namespace {

    std::string num(double value, int precision) {
        std::ostringstream out;
        out.setf(std::ios::fixed);
        out.precision(precision);
        out << value;
        std::string text = out.str();
        for (char& symbol : text) {
            if (symbol == '.') {
                symbol = ',';
            }
        }
        return text;
    }

    void write_plan(const std::filesystem::path& path, const std::vector<Trajec-
tory>& trajectories) {
        std::ofstream csv(path, std::ios::binary);
        csv << "\xEF\xBB\xBF";
        csv << "сценарий;год;культура;площадь_тыс_га;урожайность_т_га;сбор_тыс_т;"
            "выручка_млн_руб;затраты_млн_руб;маржа_млн_руб\n";
        for (const Trajectory& trajectory : trajectories) {
            for (int slot = 0; slot < N_YEARS; ++slot) {
                const SolvedYear& year = trajectory.years[slot];
                if (!year.ok) {
                    continue;
                }
                for (int i = 0; i < N_CROPS; ++i) {
                    const CropMoney money = money_of(
                        trajectory.crops[static_cast<size_t>(i)], trajec-
tory.params, year.yield[i], year.area[i]);
                    csv << trajectory.title << ';' << year.year << ';'
                        << trajectory.crops[static_cast<size_t>(i)].name << ';'
                        << num(year.area[i], 2) << ';' << num(year.yield[i], 2) <<
';'
                        << num(money.harvest_thous_t, 1) << ';' << num(money.reve-
nue_mln, 1) << ';'
                        << num(money.cost_mln, 1) << ';' << num(money.margin_mln,
1) << '\n';
                }
            }
        }
    }

    void write_trend(const std::filesystem::path& path, const TrendLine
trends[N_CROPS]) {
        std::ofstream csv(path, std::ios::binary);
        csv << "\xEF\xBB\xBF";
        csv << "культура;a;b;r2;mape_проц;урожай_2026;урожай_2027;урожай_2028\n";
        for (int i = 0; i < N_CROPS; ++i) {
            const TrendLine& line = trends[i];
            csv << crop_id(i) << ';'
                << num(line.intercept, 4) << ';' << num(line.slope, 4) << ';'
                << num(line.r2, 4) << ';' << num(line.mape, 2) << ';'
                << num(trend_at(line, 2026), 2) << ';'
                << num(trend_at(line, 2027), 2) << ';'
                << num(trend_at(line, 2028), 2) << '\n';
        }
    }
```

```
    void write_effects(const std::filesystem::path& path, const std::vector<Mar-
ginalEffect>& marginals) {
        std::ofstream csv(path, std::ios::binary);
        csv << "\xEF\xBB\xBF";
        csv << "изменение;прирост_маржи_млн_руб\n";
        for (const MarginalEffect& effect : marginals) {
            csv << effect.title << ';';
            if (effect.computed) {
                csv << num(effect.extra_margin_mln, 1);
            }
            csv << '\n';
        }
    }
}

void write_results(const std::filesystem::path& directory,
    const TrendLine trends[N_CROPS], const std::vector<Trajectory>& trajectories,
    const std::vector<MarginalEffect>& marginals) {
    std::filesystem::create_directories(directory);
    write_plan(directory / "plan.csv", trajectories);
    write_trend(directory / "trend.csv", trends);
    write_effects(directory / "effects.csv", marginals);
}
```

Файл model.cpp

```
#include "model.h"
#include "simplex.h"
#include <cmath>
#include <sstream>

double cost_per_ha(const Crop& crop, const Params& params, double yield_t_ha) {
    // Себестоимость_т * урожайность_2024 — это затраты гектара в год, когда её
мерили
    // Дальше делим их на постоянную и переменную части, переменную пересчитываем
на новый урожай
    const double cost_ha_base = crop.cost_rub_t * crop.yield_base;
    const double fixed = params.fixed_cost_share * cost_ha_base;
    const double variable = (1.0 - params.fixed_cost_share) * crop.cost_rub_t *
yield_t_ha;
    return fixed + variable;
}

double margin_per_ha(const Crop& crop, const Params& params, double yield_t_ha) {
    return crop.price_rub_t * yield_t_ha - cost_per_ha(crop, params, yield_t_ha);
}

CropMoney money_of(const Crop& crop, const Params& params, double yield_t_ha, dou-
ble area_thous_ha) {
    CropMoney money;
    money.cost_ha = cost_per_ha(crop, params, yield_t_ha);
    money.margin_ha = crop.price_rub_t * yield_t_ha - money.cost_ha;
    money.harvest_thous_t = area_thous_ha * yield_t_ha;
    // тыс. га * руб./га = тыс. руб. Деление на 1000 переводит это в млн руб
    // Тот же масштаб у сбора: тыс. т * руб./т тоже даёт тыс. руб
    money.revenue_mln = money.harvest_thous_t * crop.price_rub_t / 1000.0;
    money.cost_mln = area_thous_ha * money.cost_ha / 1000.0;
    money.margin_mln = money.revenue_mln - money.cost_mln;
    return money;
}

double total_margin(const std::vector<Crop>& crops, const Params& params,
                    const double yield[N_CROPS], const double area[N_CROPS]) {
    double sum = 0;
    for (int i = 0; i < N_CROPS; ++i) {
        sum += money_of(crops[static_cast<size_t>(i)], params, yield[i],
area[i]).margin_mln;
    }
    return sum;
}

namespace {
    // ПОДНЯТЬ ГОДОВОЙ ПОТОЛК МАЛО: годовой шаг может по-прежнему не пускать пло-
щадь до новой цифры
    // Поэтому, если нужно, увеличиваем и inertia.
    void loosen_upper(Crop& crop, double reference, double target_hi) {
        if (target_hi > crop.area_max) {
            crop.area_max = target_hi;
        }
        if (reference > 1e-6) {
            const double needed = target_hi / reference - 1.0;
            if (needed > crop.inertia) {
                crop.inertia = needed + 1e-4;
            }
        }
    }
}
```

```
SolvedYear optimize_year(const std::vector<Crop>& crops, const Params& params, int
year, const double yield[N_CROPS], const double reference[N_CROPS], double to-
tal_area) {
    SolvedYear solved;
    solved.year = year;
    for (int i = 0; i < N_CROPS; ++i) {
        solved.yield[i] = yield[i];
        solved.ref[i] = reference[i];
        if (yield[i] <= 0.05) {
            solved.error = "Слишком низкая урожайность у культуры " +
crops[static_cast<size_t>(i)].name;
            return solved;
        }

        // Рабочий отрезок — пересечение годового шага и постоянного пола/потолка
        const double by_inertia_lo = reference[i] * (1.0 -
crops[static_cast<size_t>(i)].inertia);
        const double by_inertia_hi = reference[i] * (1.0 +
crops[static_cast<size_t>(i)].inertia);
        solved.lo[i] = by_inertia_lo > crops[static_cast<size_t>(i)].area_min ?
by_inertia_lo
            : crops[static_cast<size_t>(i)].area_min;
        solved.lo_kind[i] = by_inertia_lo > crops[static_cast<size_t>(i)].area_min
? LimitKind::Inertia
            : LimitKind::Agronomic;
        solved.hi[i] = by_inertia_hi < crops[static_cast<size_t>(i)].area_max ?
by_inertia_hi
            : crops[static_cast<size_t>(i)].area_max;
        solved.hi_kind[i] = by_inertia_hi < crops[static_cast<size_t>(i)].area_max
? LimitKind::Inertia
            : LimitKind::Agronomic;
        if (i == SUGAR) {
            // Завод ограничивает тонны, а переменная задачи — гектары
            // При высоком урожае тот же объём помещается на меньшей площади
            const double by_plant = params.beet_capacity_thous_t / yield[i];
            if (by_plant < solved.hi[i]) {
                solved.hi[i] = by_plant;
                solved.hi_kind[i] = LimitKind::Capacity;
            }
        }
        if (solved.lo[i] > solved.hi[i]) {
            // Шаг просит площадь выше жёсткого потолка. Потолок оставляем, пол
подтягиваем к нему
            solved.lo[i] = solved.hi[i];
            solved.lo_kind[i] = solved.hi_kind[i];
        }
    }
    double sum_lo = 0;
    double sum_hi = 0;
    for (int i = 0; i < N_CROPS; ++i) {
        sum_lo += solved.lo[i];
        sum_hi += solved.hi[i];
    }
    if (sum_lo > total_area + 0.05) {
        solved.error = "Сумма минимальных площадей больше земельного фонда " +
std::to_string(year);
        return solved;
    }
    if (sum_hi < total_area - 0.05) {
        solved.error = "Сумма максимальных площадей меньше земельного фонда " +
std::to_string(year);
        return solved;
    }
    std::vector<LpConstraint> cons;
    {
        LpConstraint balance;
        balance.a.assign(N_CROPS, 1.0);
```

```
        balance.b = total_area;
        balance.type = LpType::EQ;
        cons.push_back(balance);
    }
    for (int i = 0; i < N_CROPS; ++i) {
        LpConstraint upper;
        upper.a.assign(N_CROPS, 0.0);
        upper.a[static_cast<size_t>(i)] = 1.0;
        upper.b = solved.hi[i];
        upper.type = LpType::LE;
        cons.push_back(upper);

        LpConstraint lower;
        lower.a.assign(N_CROPS, 0.0);
        lower.a[static_cast<size_t>(i)] = 1.0;
        lower.b = solved.lo[i];
        lower.type = LpType::GE;
        cons.push_back(lower);
    }
    {
        // x_подсолнечника - доля * x_зерна <= 0. Отдельные потолки этого не га-
рантируют:
        // ПОДЕЛЬ МОЖЕТ ОДНОВРЕМЕННО НАРСТИТЬ ПОДСОЛНЕЧНИК И ПРИ ЭТОМ СРЕЗАТЬ
ЗЕРНО!!!!!
        LpConstraint ratio;
        ratio.a.assign(N_CROPS, 0.0);
        ratio.a[SUNFLOWER] = 1.0;
        ratio.a[GRAIN] = -params.sunflower_per_grain;
        ratio.b = 0;
        ratio.type = LpType::LE;
        cons.push_back(ratio);
    }

    std::vector<double> objective(N_CROPS);
    for (int i = 0; i < N_CROPS; ++i) {
        // руб./га / 1000 * тыс. га = млн руб., поэтому значение цели совпадает с
суммой маржи
        objective[static_cast<size_t>(i)] =
            margin_per_ha(crops[static_cast<size_t>(i)], params, yield[i]) /
1000.0;
    }

    const SolveResult raw = maximize(objective, cons);
    if (raw.status == SolveStatus::Infeasible) {
        solved.error = "Симплекс: ограничения " + std::to_string(year) + "
несовместны";
        return solved;
    }
    if (raw.status != SolveStatus::Optimal) {
        solved.error = "Симплекс (" + std::to_string(year) + "): " + raw.message;
        return solved;
    }

    double area_sum = 0;
    for (int i = 0; i < N_CROPS; ++i) {
        solved.area[i] = raw.x[static_cast<size_t>(i)];
        area_sum += solved.area[i];
        if (solved.area[i] < solved.lo[i] - 0.05 || solved.area[i] > solved.hi[i]
+ 0.05) {
            solved.error = "Решение вышло за границу площади: " +
crops[static_cast<size_t>(i)].name;
            return solved;
        }
    }
    if (std::fabs(area_sum - total_area) > 0.05) {
        solved.error = "Сумма площадей не сошлась с земельным фондом";
        return solved;
```

```
    }
    const double sun_limit = params.sunflower_per_grain * solved.area[GRAIN];
    if (solved.area[SUNFLOWER] > sun_limit + 0.05) {
        solved.error = "Нарушено соотношение подсолнечника и зерновых";
        return solved;
    }
    solved.ratio_active = std::fabs(solved.area[SUNFLOWER] - sun_limit) <= 0.08;
    // Та же маржа, посчитанная формулами денег, а не симплекс-таблицей
    // Расхождение больше 0,5 млн руб - перепутаны единицы или знак цели
    const double recounted = total_margin(crops, params, solved.yield,
solved.area);
    if (std::fabs(recounted - raw.objective) > 0.5) {
        std::ostringstream text;
        text << "Маржа симплекса (" << raw.objective << ") не сошлась с пересчётом
(" << recounted << ")";
        solved.error = text.str();
        return solved;
    }
    solved.ok = true;
    return solved;
}

Trajectory run_trajectory(const std::string& key, const std::string& title,
    const std::vector<Crop>& crops, const Params& params, double total_area,
    const double yields[N_YEARS][N_CROPS]) {
    Trajectory trajectory;
    trajectory.key = key;
    trajectory.title = title;
    trajectory.crops = crops;
    trajectory.params = params;
    double reference[N_CROPS];
    for (int i = 0; i < N_CROPS; ++i) {
        reference[i] = crops[static_cast<size_t>(i)].area_2025;
    }
    trajectory.ok = true;
    for (int slot = 0; slot < N_YEARS; ++slot) {
        trajectory.years[slot] =
            optimize_year(crops, params, PLAN_YEARS[slot], yields[slot], refer-
ence, total_area);
        if (!trajectory.years[slot].ok) {
            trajectory.ok = false;
            trajectory.error = trajectory.years[slot].error;
            break;
        }
        // Следующий год отталкивается от уже выбранного плана, а не снова от
факта 2025 года.
        for (int i = 0; i < N_CROPS; ++i) {
            reference[i] = trajectory.years[slot].area[i];
        }
    }
    return trajectory;
}

std::vector<MarginalEffect> marginal_effects(const std::vector<Crop>& crops, const
Params& params, const SolvedYear& year, double total_area) {
    std::vector<MarginalEffect> effects;
    if (!year.ok) {
        return effects;
    }
    const double base = total_margin(crops, params, year.yield, year.area);
    struct Spec {
        const char* title;
        const char* meaning;
        int kind;
    };
    const Spec specs[] = {
        {"Картофель, потолок площади +1 тыс. га",
```

```
         "Хранилища, техника и люди, чтобы за год добавить тысячу гектаров карто-
феля.",
         0},
        {"Овощи, потолок площади +0,5 тыс. га",
         "То же для овощей открытого грунта: хранение, полив, рабочие руки.",
         1},
        {"Сахарные заводы, мощность +100 тыс. т",
         "Если заводы смогут принять на 100 тыс. т свёклы больше.",
         2},
        {"Севооборот подсолнечника 0,40 -> 0,45 к зерну",
         "Если допустить более плотный возврат подсолнечника на те же поля.",
         3},
    };
    for (const Spec& spec : specs) {
        MarginalEffect effect;
        effect.title = spec.title;
        effect.meaning = spec.meaning;
        std::vector<Crop> tweaked = crops;
        Params tweaked_params = params;
        if (spec.kind == 0) {
            loosen_upper(tweaked[POTATO], year.ref[POTATO], year.area[POTATO] +
1.0);
        } else if (spec.kind == 1) {
            loosen_upper(tweaked[VEGETABLES], year.ref[VEGETABLES], year.area[VEG-
ETABLES] + 0.5);
        } else if (spec.kind == 2) {
            tweaked_params.beet_capacity_thous_t += 100.0;
        } else {
            tweaked_params.sunflower_per_grain += 0.05;
        }
        const SolvedYear variant =
            optimize_year(tweaked, tweaked_params, year.year, year.yield,
year.ref, total_area);
        if (!variant.ok) {
            effect.error = variant.error;
            effects.push_back(effect);
            continue;
        }
        effect.computed = true;
        effect.extra_margin_mln = total_margin(tweaked, tweaked_params, vari-
ant.yield, variant.area) - base;
        effects.push_back(effect);
    }
    return effects;
}
```

Файл forecast.cpp

```
#include "forecast.h"
#include <cmath>
#include <stdexcept>

TrendLine fit_trend(const std::vector<YearRow>& history, int crop, int year_from,
int year_to) {
    double sum_x = 0;
    double sum_y = 0;
    double sum_xx = 0;
    double sum_xy = 0;
    int n = 0;
    for (const YearRow& row : history) {
        if (row.year < year_from || row.year > year_to) {
            continue;
        }
        // 0 оси X — первый год окна, тогда intercept читается как урожайность
year_from
        const double x = static_cast<double>(row.year - year_from);
        const double y = row.yield[crop];
        sum_x += x;
        sum_y += y;
        sum_xx += x * x;
        sum_xy += x * y;
        ++n;
    }
    if (n < 3) {
        throw std::runtime_error("Для линейного тренда нужно хотя бы три года");
    }
    const double nd = static_cast<double>(n);
    const double denom = nd * sum_xx - sum_x * sum_x;
    if (std::fabs(denom) < 1e-12) {
        throw std::runtime_error("Не удалось оценить наклон тренда");
    }

    TrendLine line;
    line.n = n;
    line.origin = year_from;
    // Обычные формулы МНК для y = a + b*x, без матриц: точек всего 15
    line.slope = (nd * sum_xy - sum_x * sum_y) / denom;
    line.intercept = (sum_y - line.slope * sum_x) / nd;
    double ss_tot = 0;
    double ss_res = 0;
    double mape_sum = 0;
    int mape_n = 0;
    const double mean_y = sum_y / nd;
    for (const YearRow& row : history) {
        if (row.year < year_from || row.year > year_to) {
            continue;
        }
        const double y = row.yield[crop];
        const double fit = trend_at(line, row.year);
        const double err = y - fit;
        ss_res += err * err;
        const double dev = y - mean_y;
        ss_tot += dev * dev;
        if (std::fabs(y) > 1e-9) {
            mape_sum += std::fabs(err / y);
            ++mape_n;
        }
    }
    // R2 = 1 - (разброс остатков / разброс самого ряда). MAPE — средняя |ошибка|
/ |факт|, в "%"
```

```
    line.r2 = ss_tot < 1e-12 ? 1.0 : 1.0 - ss_res / ss_tot;
    line.mape = mape_n ? 100.0 * mape_sum / static_cast<double>(mape_n) : 0;
    return line;
}

double trend_at(const TrendLine& line, int year) {
    return line.intercept + line.slope * static_cast<double>(year - line.origin);
}
```

Файл data.cpp

```
#include "data.h"
#include <cmath>
#include <fstream>
#include <map>
#include <sstream>

namespace {
    std::string trim(std::string text) {
        while (!text.empty() && (text.back() == '\r' || text.back() == ' ' ||
text.back() == '\t')) {
            text.pop_back();
        }
        size_t begin = 0;
        while (begin < text.size() && (text[begin] == ' ' || text[begin] == '\t'))
{
            ++begin;
        }
        if (begin) {
            text.erase(0, begin);
        }
        // Excel ИНОГДА СОХРАНЕТ CSV С НЕВИДИМОЙ МЕТКОЙ UTF-8 В НАЧАЛЕ!!!!
        // ЕСЛИ ЕЕ НЕ СНЯТЬ, СТОБЕЦ year НЕ НАЙДЕТСЯ!!!!!!
        if (text.size() >= 3 &&
            static_cast<unsigned char>(text[0]) == 0xEF &&
            static_cast<unsigned char>(text[1]) == 0xBB &&
            static_cast<unsigned char>(text[2]) == 0xBF) {
            text.erase(0, 3);
        }
        return text;
    }

    std::vector<std::string> split_semicolon(const std::string& line) {
        std::vector<std::string> parts;
        std::string current;
        for (char ch : line) {
            if (ch == ';') {
                parts.push_back(trim(current));
                current.clear();
            }
            else {
                current.push_back(ch);
            }
        }
        parts.push_back(trim(current));
        return parts;
    }

    double to_double(const std::string& text, const std::string& where) {
        try {
            size_t used = 0;
            const double value = std::stod(text, &used);
            if (used != text.size()) {
                throw std::runtime_error("хвост");
            }
            return value;
        }
        catch (const std::exception&) {
            throw std::runtime_error("Не число (" + text + ") в " + where);
        }
    }

    int to_int(const std::string& text, const std::string& where) {
        try {
```

```
            size_t used = 0;
            const int value = std::stoi(text, &used);
            if (used != text.size()) {
                throw std::runtime_error("хвост");
            }
            return value;
        }
        catch (const std::exception&) {
            throw std::runtime_error("Не целое (" + text + ") в " + where);
        }
    }

    std::filesystem::path find_file(const std::vector<std::filesystem::path>&
roots, const std::string& name) {
        for (const std::filesystem::path& root : roots) {
            const std::filesystem::path direct = root / name;
            if (std::filesystem::exists(direct)) {
                return direct;
            }
            const std::filesystem::path nested = root / "data" / name;
            if (std::filesystem::exists(nested)) {
                return nested;
            }
        }
        throw std::runtime_error(
            "Не найден файл data/" + name +
            ". Запускайте программу из папки проекта");
    }

    std::vector<std::string> read_lines(const std::filesystem::path& path) {
        std::ifstream input(path);
        if (!input) {
            throw std::runtime_error("Не удалось открыть " + path.file-
name().string());
        }
        std::vector<std::string> lines;
        std::string line;
        while (std::getline(input, line)) {
            line = trim(line);
            if (line.empty() || line[0] == '#') {
                continue;
            }
            lines.push_back(line);
        }
        return lines;
    }
}

InputData load_input(const std::vector<std::filesystem::path>& roots) {
    InputData data;
    data.crops.assign(N_CROPS, Crop{});
    const auto economy_lines = read_lines(find_file(roots, "economics.csv"));
    bool seen[N_CROPS]{};
    for (const std::string& line : economy_lines) {
        const std::vector<std::string> parts = split_semicolon(line);
        if (parts.empty()) {
            continue;
        }
        if (parts[0] == "param") {
            if (parts.size() != 3) {
                throw std::runtime_error("Строка param в economics.csv должна
содержать 3 поля");
            }
            const double value = to_double(parts[2], "economics.csv");
            if (parts[1] == "fixed_cost_share") {
                data.params.fixed_cost_share = value;
            } else if (parts[1] == "beet_capacity_thous_t") {
```

```
                data.params.beet_capacity_thous_t = value;
            } else if (parts[1] == "sunflower_per_grain") {
                data.params.sunflower_per_grain = value;
            } else {
                throw std::runtime_error("Неизвестный параметр " + parts[1]);
            }
            continue;
        }
        if (parts[0] != "crop" || parts.size() != 10) {
            throw std::runtime_error("Непонятная строка в economics.csv: " +
line);
        }
        const int index = crop_index(parts[1]);
        if (index < 0) {
            throw std::runtime_error("Неизвестная культура " + parts[1]);
        }
        Crop& crop = data.crops[static_cast<size_t>(index)];
        crop.id = parts[1];
        crop.name = parts[2];
        crop.price_rub_t = to_double(parts[3], crop.id);
        crop.cost_rub_t = to_double(parts[4], crop.id);
        crop.yield_base = to_double(parts[5], crop.id);
        crop.area_2025 = to_double(parts[6], crop.id);
        crop.area_min = to_double(parts[7], crop.id);
        crop.area_max = to_double(parts[8], crop.id);
        crop.inertia = to_double(parts[9], crop.id);
        if (crop.price_rub_t <= 0 || crop.cost_rub_t <= 0 || crop.yield_base <= 0)
{
            throw std::runtime_error("Цена, себестоимость и базовая урожайность
должны быть больше нуля: " + crop.id);
        }
        if (crop.area_min > crop.area_2025 || crop.area_2025 > crop.area_max) {
            throw std::runtime_error("Площадь 2025 года вне границ: " +
crop.name);
        }
        if (crop.inertia < 0 || crop.inertia > 1.0) {
            throw std::runtime_error("Шаг inertia должен быть от 0 до 1: " +
crop.name);
        }
        seen[index] = true;
        data.total_area += crop.area_2025;
    }
    for (int i = 0; i < N_CROPS; ++i) {
        if (!seen[i]) {
            throw std::runtime_error(std::string("В economics.csv нет культуры ")
+ crop_id(i));
        }
    }
    if (data.params.fixed_cost_share < 0 || data.params.fixed_cost_share > 1) {
        throw std::runtime_error("fixed_cost_share должен быть от 0 до 1");
    }
    if (data.params.beet_capacity_thous_t <= 0 || data.params.sunflower_per_grain
<= 0) {
        throw std::runtime_error("Мощность заводов и доля подсолнечника должны
быть больше нуля");
    }
    if (data.crops[SUNFLOWER].area_2025 >
        data.params.sunflower_per_grain * data.crops[GRAIN].area_2025 + 0.05) {
        throw std::runtime_error("Факт 2025 года уже нарушает соотношение подсол-
нечника и зерновых");
    }

    const auto history_lines = read_lines(find_file(roots, "yield_area.csv"));
    if (history_lines.size() < 2) {
        throw std::runtime_error("yield_area.csv пустой");
    }
    const std::vector<std::string> header = split_semicolon(history_lines[0]);
```

```
    std::map<std::string, int> column;
    for (int i = 0; i < static_cast<int>(header.size()); ++i) {
        column[header[static_cast<size_t>(i)]] = i;
    }
    const char* required[] = {
        "year", "p_grain", "p_sugar", "p_sunflower", "p_potato", "p_vegetables",
        "s_grain", "s_sugar", "s_sunflower", "s_potato", "s_vegetables"
    };
    for (const char* name : required) {
        if (!column.count(name)) {
            throw std::runtime_error(std::string("В yield_area.csv нет столбца ")
+ name);
        }
    }
    const int yield_col[N_CROPS] = {
        column["p_grain"], column["p_sugar"], column["p_sunflower"], column["p_po-
tato"], column["p_vegetables"]
    };
    const int area_col[N_CROPS] = {
        column["s_grain"], column["s_sugar"], column["s_sunflower"], column["s_po-
tato"], column["s_vegetables"]
    };

    for (size_t line_i = 1; line_i < history_lines.size(); ++line_i) {
        const std::vector<std::string> parts = split_semicolon(his-
tory_lines[line_i]);
        if (static_cast<int>(parts.size()) < static_cast<int>(header.size())) {
            throw std::runtime_error("Короткая строка в yield_area.csv, год около
строки " + std::to_string(line_i + 1));
        }
        YearRow row;
        row.year = to_int(parts[static_cast<size_t>(column["year"])],
"yield_area.csv");
        for (int crop = 0; crop < N_CROPS; ++crop) {
            row.yield[crop] = to_dou-
ble(parts[static_cast<size_t>(yield_col[crop])], "yield_area.csv");
            row.area[crop] = to_double(parts[static_cast<size_t>(area_col[crop])],
"yield_area.csv");
        }
        data.history.push_back(row);
    }

    const YearRow* year_2024 = nullptr;
    const YearRow* year_2025 = nullptr;
    for (const YearRow& row : data.history) {
        if (row.year == 2024) {
            year_2024 = &row;
        }
        if (row.year == 2025) {
            year_2025 = &row;
        }
    }
    if (!year_2024 || !year_2025) {
        throw std::runtime_error("В ряде нет 2024 или 2025 года");
    }
    for (int i = 0; i < N_CROPS; ++i) {
        // Затраты считаются от урожайности 2024 года, площади плана — от 2025-го
        // Если 2 файла разъехались, МАРЖА И "к 2025" В ОТЧЕТЕ БУДУТ ПРО РАЗНЫЕ
ВЕЩИ!!!!!
        if (std::fabs(year_2024->yield[i] -
data.crops[static_cast<size_t>(i)].yield_base) > 0.02) {
            throw std::runtime_error("Базовая урожайность в economics.csv не
совпала с 2024 годом: " +
                data.crops[static_cast<size_t>(i)].name);
        }
        if (std::fabs(year_2025->area[i] -
data.crops[static_cast<size_t>(i)].area_2025) > 0.2) {
```

```
            throw std::runtime_error("Площадь 2025 года в economics.csv не совпала
с рядом: " +
                data.crops[static_cast<size_t>(i)].name);
        }
    }

    bool got[N_CROPS][N_YEARS]{};
    const auto forecast_lines = read_lines(find_file(roots, "forecast_boot-
strap.csv"));
    for (const std::string& line : forecast_lines) {
        const std::vector<std::string> parts = split_semicolon(line);
        if (parts.size() != 5) {
            throw std::runtime_error("Строка прогноза должна содержать 5 полей: "
+ line);
        }
        const int index = crop_index(parts[0]);
        const int year = to_int(parts[1], "forecast_bootstrap.csv");
        if (index < 0 || year < 2026 || year > 2028) {
            throw std::runtime_error("Плохой ключ прогноза: " + line);
        }
        const int slot = year - 2026;
        Band& band = data.bootstrap[index][slot];
        band.low = to_double(parts[2], "forecast_bootstrap.csv");
        band.median = to_double(parts[3], "forecast_bootstrap.csv");
        band.high = to_double(parts[4], "forecast_bootstrap.csv");
        if (!(band.low <= band.median && band.median <= band.high) || band.low <=
0) {
            throw std::runtime_error("Интервал прогноза перепутан: " + line);
        }
        got[index][slot] = true;
    }
    for (int crop = 0; crop < N_CROPS; ++crop) {
        for (int year = 0; year < N_YEARS; ++year) {
            if (!got[crop][year]) {
                throw std::runtime_error(std::string("Нет прогноза ") +
crop_id(crop) + " на " + std::to_string(2026 + year));
            }
        }
    }
    return data;
}
```

Файл main.cpp

```
#include "data.h"
#include "forecast.h"
#include "model.h"
#include "report.h"
#include "simplex.h"
#define NOMINMAX
#include <windows.h>
#include <knownfolders.h>
#include <objbase.h>
#include <shellapi.h>
#include <shlobj.h>
#include <filesystem>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

    constexpr int ID_OPEN_DIR = 101;
    constexpr int ID_OPEN_PLAN = 102;
    constexpr int ID_CLOSE = 103;
    constexpr int HEADER_PX = 92;
    struct Outcome {
        bool ok = false;
        std::wstring text;
        std::filesystem::path directory;
    };
    HWND g_edit = nullptr;
    HWND g_open_dir = nullptr;
    HWND g_open_plan = nullptr;
    HWND g_close = nullptr;
    HFONT g_title_font = nullptr;
    HFONT g_sub_font = nullptr;
    HFONT g_text_font = nullptr;
    HFONT g_button_font = nullptr;
    Outcome g_outcome;

    std::wstring widen(const std::string& utf8) {
        if (utf8.empty()) {
            return {};
        }
        const int size = MultiByteToWideChar(CP_UTF8, 0, utf8.data(),
static_cast<int>(utf8.size()), nullptr, 0);
        std::wstring wide(static_cast<size_t>(size), L'\0');
        MultiByteToWideChar(CP_UTF8, 0, utf8.data(),
static_cast<int>(utf8.size()), wide.data(), size);
        return wide;
    }

    int dpi_of(HWND hwnd) {
        const UINT dpi = GetDpiForWindow(hwnd);
        return dpi == 0 ? 96 : static_cast<int>(dpi);
    }

    int scale_px(HWND hwnd, int px_at_96) {
        return MulDiv(px_at_96, dpi_of(hwnd), 96);
    }

    std::filesystem::path executable_directory() {
        wchar_t buffer[MAX_PATH];
        const DWORD length = GetModuleFileNameW(nullptr, buffer, MAX_PATH);
        if (length == 0 || length >= MAX_PATH) {
```

```
            return {};
        }
        return std::filesystem::path(buffer).parent_path();
    }
    std::filesystem::path results_directory() {
        std::error_code ec;
        const std::filesystem::path cwd = std::filesystem::current_path();
        if (std::filesystem::exists(cwd / "PenzaCropOpt.sln", ec)) {
            return cwd / "results";
        }
        PWSTR documents = nullptr;
        if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Documents, 0, nullptr, &docu-
ments))) {
            const std::filesystem::path path = std::filesystem::path(documents) /
L"Оптимизация посевных площадей";
            CoTaskMemFree(documents);
            return path;
        }
        return executable_directory() / "results";
    }

    // band_field: -1 низ 80% интервала, 0 медиана, +1 верх. Три сценария урожай-
ности собираются одним проходом по таблице 10
    void fill_bootstrap(const InputData& data, int band_field, double
yields[N_YEARS][N_CROPS]) {
        for (int slot = 0; slot < N_YEARS; ++slot) {
            for (int crop = 0; crop < N_CROPS; ++crop) {
                const Band& band = data.bootstrap[crop][slot];
                if (band_field < 0) {
                    yields[slot][crop] = band.low;
                }
                else if (band_field > 0) {
                    yields[slot][crop] = band.high;
                }
                else {
                    yields[slot][crop] = band.median;
                }
            }
        }
    }

    std::wstring decimal(double value, int precision) {
        std::ostringstream out;
        out.setf(std::ios::fixed);
        out.precision(precision);
        out << value;
        std::string text = out.str();
        for (char& symbol : text) {
            if (symbol == '.') {
                symbol = ',';
            }
        }
        return widen(text);
    }

    std::wstring right(const std::wstring& text, int width) {
        if (static_cast<int>(text.size()) >= width) {
            return text;
        }
        return std::wstring(static_cast<size_t>(width - text.size()), L' ') +
text;
    }

    std::wstring grouped(double value) {
        long long number = std::llround(value);
        const bool negative = number < 0;
        if (negative) {
```

```
            number = -number;
        }
        const std::wstring digits = std::to_wstring(number);
        std::wstring text;
        int count = 0;
        for (int i = static_cast<int>(digits.size()) - 1; i >= 0; --i) {
            if (count > 0 && count % 3 == 0) {
                text.insert(text.begin(), L' ');
            }
            text.insert(text.begin(), digits[static_cast<size_t>(i)]);
            ++count;
        }
        if (negative) {
            text.insert(text.begin(), L'-');
        }
        return text;
    }

    double plan_margin(const Trajectory& trajectory, int slot) {
        const SolvedYear& year = trajectory.years[slot];
        double sum = 0;
        for (int i = 0; i < N_CROPS; ++i) {
            sum += money_of(trajectory.crops[static_cast<size_t>(i)], trajec-
tory.params,
                year.yield[i], year.area[i]).margin_mln;
        }
        return sum;
    }

    double fact_margin(const Trajectory& trajectory, int slot) {
        double area[N_CROPS];
        for (int i = 0; i < N_CROPS; ++i) {
            area[i] = trajectory.crops[static_cast<size_t>(i)].area_2025;
        }
        return total_margin(trajectory.crops, trajectory.params, trajec-
tory.years[slot].yield, area);
    }

    const Trajectory* find_key(const std::vector<Trajectory>& trajectories, const
char* key) {
        for (const Trajectory& trajectory : trajectories) {
            if (trajectory.key == key) {
                return &trajectory;
            }
        }
        return nullptr;
    }

    std::wstring build_summary(const std::vector<Trajectory>& trajectories,
        const std::vector<MarginalEffect>& marginals, const std::filesystem::path&
directory) {
        std::wstring text = L"Расчёт выполнен.\r\n\r\n";
        const Trajectory* base = find_key(trajectories, "base");
        const wchar_t* labels[N_CROPS] = {
            L"Зерно", L"Свёкла", L"Подсолнечник", L"Картофель", L"Овощи"
        };
        if (base != nullptr && base->ok) {
            text += L"Базовый сценарий. Площади, тыс. га\r\n";
            text += L"                2026     2027     2028\r\n";
            for (int i = 0; i < N_CROPS; ++i) {
                text += right(labels[i], 14);
                for (int slot = 0; slot < N_YEARS; ++slot) {
                    text += L"  " + right(decimal(base->years[slot].area[i], 1),
6);
                }
                text += L"\r\n";
            }
```

```
            text += L"\r\nМаржа, млн руб.\r\n";
            text += L"        план    площади 2025     выигрыш\r\n";
            double gain_sum = 0;
            for (int slot = 0; slot < N_YEARS; ++slot) {
                const double plan = plan_margin(*base, slot);
                const double fact = fact_margin(*base, slot);
                const double gain = plan - fact;
                gain_sum += gain;
                text += std::to_wstring(PLAN_YEARS[slot]);
                text += L"  " + right(grouped(plan), 8);
                text += L"  " + right(grouped(fact), 12);
                text += L"  " + right(grouped(gain), 10);
                text += L"\r\n";
            }
            text += L"3 года" + right(grouped(gain_sum), 34) + L"\r\n";
        }

        text += L"\r\nВыигрыш остальных сценариев за 3 года, млн руб.\r\n";
        for (const Trajectory& trajectory : trajectories) {
            if (trajectory.key == "base") {
                continue;
            }
            text += trajectory.title.empty() ? widen(trajectory.key) : widen(tra-
jectory.title);
            if (!trajectory.ok) {
                text += L": не посчитан\r\n";
                continue;
            }
            double gain = 0;
            for (int slot = 0; slot < N_YEARS; ++slot) {
                if (trajectory.years[slot].ok) {
                    gain += plan_margin(trajectory, slot) - fact_margin(trajec-
tory, slot);
                }
            }
            text += L"  " + grouped(gain) + L"\r\n";
        }

        if (!marginals.empty()) {
            text += L"\r\nПрирост маржи 2028 при одном ослабленном ограничении,
млн руб.\r\n";
            for (const MarginalEffect& effect : marginals) {
                text += widen(effect.title) + L"  ";
                text += effect.computed ? decimal(effect.extra_margin_mln, 1) :
L"—";
                text += L"\r\n";
            }
        }

        text += L"\r\nФайлы\r\n";
        text += (directory / L"plan.csv").wstring() + L"\r\n";
        text += (directory / L"trend.csv").wstring() + L"\r\n";
        text += (directory / L"effects.csv").wstring();
        return text;
    }

    Outcome calculate() {
        Outcome outcome;
        std::string test_message;
        if (!simplex_self_test(test_message)) {
            outcome.text = L"Симплекс-метод не прошёл контрольные задачи.\r\n" +
widen(test_message);
            return outcome;
        }
        try {
            std::vector<std::filesystem::path> roots;
            roots.push_back(std::filesystem::current_path());
```

```
            const std::filesystem::path exe_dir = executable_directory();
            if (!exe_dir.empty()) {
                roots.push_back(exe_dir);
                if (exe_dir.has_parent_path()) {
                    roots.push_back(exe_dir.parent_path());
                }
            }

            const InputData data = load_input(roots);
            TrendLine trends[N_CROPS];
            for (int crop = 0; crop < N_CROPS; ++crop) {
                trends[crop] = fit_trend(data.history, crop, 2011, 2025);
            }

            double base_yield[N_YEARS][N_CROPS];
            double high_yield[N_YEARS][N_CROPS];
            double low_yield[N_YEARS][N_CROPS];
            double trend_yield[N_YEARS][N_CROPS];
            fill_bootstrap(data, 0, base_yield);
            fill_bootstrap(data, 1, high_yield);
            fill_bootstrap(data, -1, low_yield);

            bool trend_clamped = false;
            for (int slot = 0; slot < N_YEARS; ++slot) {
                for (int crop = 0; crop < N_CROPS; ++crop) {
                    double value = trend_at(trends[crop], PLAN_YEARS[slot]);
                    // Прямая на коротком ряде МОЖЕТ УЙТИ НИЖЕ 0. В задачу такой
урожай не кладём:
                    // ЛОМАЕТСЯ ПОЧТИ ВСЕ:  и маржа, и перевод мощности завода из
тонн в гектары
                    if (value < 0.2) {
                        value = 0.2;
                        trend_clamped = true;
                    }
                    trend_yield[slot][crop] = value;
                }
            }

            std::vector<Trajectory> trajectories;
            trajectories.push_back(run_trajectory(
                "base", "Базовый, медиана прогноза", data.crops, data.params,
data.total_area, base_yield));
            trajectories.push_back(run_trajectory(
                "high", "Оптимистичный, верх 80% интервала", data.crops,
data.params, data.total_area, high_yield));
            trajectories.push_back(run_trajectory(
                "low", "Пессимистичный, низ 80% интервала", data.crops,
data.params, data.total_area, low_yield));
            Trajectory trend = run_trajectory(
                "trend", "Контрольный, линейный тренд", data.crops, data.params,
data.total_area, trend_yield);
            if (trend_clamped) {
                trend.note = "Отрицательный тренд обрезан снизу значением 0,2
т/га.";
            }
            trajectories.push_back(trend);

            // Урожай оставляем базовым, дешевеют только картофель и овощи
            // Так видно, держится ли их расширение на высокой цене 2025 года
            std::vector<Crop> stressed = data.crops;
            stressed[POTATO].price_rub_t *= 0.7;
            stressed[VEGETABLES].price_rub_t *= 0.7;
            Trajectory stress = run_trajectory(
                "stress", "Ценовой стресс, картофель и овощи -30%", stressed,
data.params, data.total_area, base_yield);
            stress.note = "Урожайность базовая. Проверяется, не развалится ли план
при дешёвом картофеле и овощах.";
```

```
            trajectories.push_back(stress);
            std::vector<MarginalEffect> marginals;
            bool failed = false;
            for (const Trajectory& trajectory : trajectories) {
                if (!trajectory.ok) {
                    failed = true;
                }
                if (trajectory.key == "base" && trajectory.ok) {
                    marginals = marginal_effects(data.crops, data.params, trajec-
tory.years[N_YEARS - 1], data.total_area);
                }
            }

            outcome.directory = results_directory();
            write_results(outcome.directory, trends, trajectories, marginals);
            outcome.ok = !failed;
            outcome.text = build_summary(trajectories, marginals, outcome.direc-
tory);
            if (failed) {
                outcome.text += L"\r\n\r\nОдин из сценариев не решён.";
                for (const Trajectory& trajectory : trajectories) {
                    if (!trajectory.ok) {
                        outcome.text += L"\r\n" + widen(trajectory.title) + L": "
+ widen(trajectory.error);
                    }
                }
            }
            return outcome;
        } catch (const std::exception& error) {
            outcome.text = L"Расчёт остановился.\r\n" + widen(error.what());
            return outcome;
        }
    }

    void drop_fonts() {
        if (g_title_font != nullptr) {
            DeleteObject(g_title_font);
            g_title_font = nullptr;
        }
        if (g_sub_font != nullptr) {
            DeleteObject(g_sub_font);
            g_sub_font = nullptr;
        }
        if (g_text_font != nullptr) {
            DeleteObject(g_text_font);
            g_text_font = nullptr;
        }
        if (g_button_font != nullptr) {
            DeleteObject(g_button_font);
            g_button_font = nullptr;
        }
    }

    HFONT make_font(HWND hwnd, int px, int weight, const wchar_t* face) {
        return CreateFontW(-scale_px(hwnd, px), 0, 0, 0, weight, FALSE, FALSE,
FALSE, RUSSIAN_CHARSET,
            OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DE-
FAULT_PITCH | FF_DONTCARE, face);
    }

    void apply_fonts(HWND hwnd) {
        drop_fonts();
        g_title_font = make_font(hwnd, 26, FW_SEMIBOLD, L"Segoe UI");
        g_sub_font = make_font(hwnd, 15, FW_NORMAL, L"Segoe UI");
        g_text_font = make_font(hwnd, 16, FW_NORMAL, L"Consolas");
        g_button_font = make_font(hwnd, 15, FW_NORMAL, L"Segoe UI");
```

```
        SendMessageW(g_edit, WM_SETFONT, reinterpret_cast<WPARAM>(g_text_font),
TRUE);
        SendMessageW(g_open_dir, WM_SETFONT, reinterpret_cast<WPARAM>(g_but-
ton_font), TRUE);
        SendMessageW(g_open_plan, WM_SETFONT, reinterpret_cast<WPARAM>(g_but-
ton_font), TRUE);
        SendMessageW(g_close, WM_SETFONT, reinterpret_cast<WPARAM>(g_button_font),
TRUE);
    }

    void layout_window(HWND hwnd) {
        RECT client{};
        GetClientRect(hwnd, &client);
        const int margin = scale_px(hwnd, 16);
        const int header = scale_px(hwnd, HEADER_PX);
        const int button_h = scale_px(hwnd, 34);
        const int button_gap = scale_px(hwnd, 10);
        const int bottom = client.bottom - margin - button_h;
        const int edit_top = header + margin;
        MoveWindow(g_edit, margin, edit_top, client.right - 2 * margin, bottom -
edit_top - margin, TRUE);

        const int close_w = scale_px(hwnd, 110);
        const int plan_w = scale_px(hwnd, 170);
        const int dir_w = scale_px(hwnd, 210);
        int x = client.right - margin - close_w;
        MoveWindow(g_close, x, bottom, close_w, button_h, TRUE);
        x -= button_gap + plan_w;
        MoveWindow(g_open_plan, x, bottom, plan_w, button_h, TRUE);
        x -= button_gap + dir_w;
        MoveWindow(g_open_dir, x, bottom, dir_w, button_h, TRUE);
    }

    void open_path(const std::filesystem::path& path) {
        if (path.empty()) {
            return;
        }
        ShellExecuteW(nullptr, L"open", path.c_str(), nullptr, nullptr,
SW_SHOWNORMAL);
    }

    LRESULT CALLBACK window_proc(HWND hwnd, UINT message, WPARAM wparam, LPARAM
lparam) {
        switch (message) {
        case WM_CREATE: {
            g_edit = CreateWindowExW(WS_EX_CLIENTEDGE, L"EDIT", L"",
                WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_MULTILINE | ES_READONLY |
ES_AUTOVSCROLL,
                0, 0, 0, 0, hwnd, nullptr, nullptr, nullptr);
            g_open_dir = CreateWindowExW(0, L"BUTTON", L"Открыть папку",
                WS_CHILD | WS_VISIBLE | WS_TABSTOP, 0, 0, 0, 0, hwnd,
                reinterpret_cast<HMENU>(static_cast<INT_PTR>(ID_OPEN_DIR)),
nullptr, nullptr);
            g_open_plan = CreateWindowExW(0, L"BUTTON", L"Открыть plan.csv",
                WS_CHILD | WS_VISIBLE | WS_TABSTOP, 0, 0, 0, 0, hwnd,
                reinterpret_cast<HMENU>(static_cast<INT_PTR>(ID_OPEN_PLAN)),
nullptr, nullptr);
            g_close = CreateWindowExW(0, L"BUTTON", L"Закрыть",
                WS_CHILD | WS_VISIBLE | WS_TABSTOP, 0, 0, 0, 0, hwnd,
                reinterpret_cast<HMENU>(static_cast<INT_PTR>(ID_CLOSE)), nullptr,
nullptr);
            apply_fonts(hwnd);
            SetWindowTextW(g_edit, g_outcome.text.c_str());
            const BOOL enable = g_outcome.ok ? TRUE : FALSE;
            EnableWindow(g_open_dir, enable);
            EnableWindow(g_open_plan, enable);
            layout_window(hwnd);
```

```
            return 0;
        }
        case WM_SIZE:
            layout_window(hwnd);
            return 0;
        case WM_GETMINMAXINFO: {
            MINMAXINFO* info = reinterpret_cast<MINMAXINFO*>(lparam);
            info->ptMinTrackSize.x = scale_px(hwnd, 760);
            info->ptMinTrackSize.y = scale_px(hwnd, 520);
            return 0;
        }
        case WM_DPICHANGED: {
            const RECT* suggested = reinterpret_cast<RECT*>(lparam);
            SetWindowPos(hwnd, nullptr, suggested->left, suggested->top,
                suggested->right - suggested->left, suggested->bottom - suggested-
>top,
                SWP_NOZORDER | SWP_NOACTIVATE);
            apply_fonts(hwnd);
            layout_window(hwnd);
            return 0;
        }
        case WM_COMMAND:
            switch (LOWORD(wparam)) {
            case ID_OPEN_DIR:
                open_path(g_outcome.directory);
                return 0;
            case ID_OPEN_PLAN:
                open_path(g_outcome.directory / L"plan.csv");
                return 0;
            case ID_CLOSE:
                DestroyWindow(hwnd);
                return 0;
            default:
                break;
            }
            break;
        case WM_PAINT: {
            PAINTSTRUCT paint{};
            HDC dc = BeginPaint(hwnd, &paint);
            RECT client{};
            GetClientRect(hwnd, &client);
            RECT header = client;
            header.bottom = scale_px(hwnd, HEADER_PX);
            HBRUSH green = CreateSolidBrush(RGB(31, 107, 58));
            FillRect(dc, &header, green);
            DeleteObject(green);
            RECT body = client;
            body.top = header.bottom;
            HBRUSH paper = CreateSolidBrush(RGB(244, 246, 243));
            FillRect(dc, &body, paper);
            DeleteObject(paper);

            SetBkMode(dc, TRANSPARENT);
            SetTextColor(dc, RGB(255, 255, 255));
            RECT title = header;
            title.left += scale_px(hwnd, 18);
            title.top += scale_px(hwnd, 16);
            title.bottom = title.top + scale_px(hwnd, 34);
            SelectObject(dc, g_title_font);
            DrawTextW(dc, L"Оптимизация посевных площадей", -1, &title, DT_LEFT |
DT_SINGLELINE | DT_VCENTER);
            RECT subtitle = header;
            subtitle.left += scale_px(hwnd, 18);
            subtitle.top = title.bottom;
            subtitle.bottom = header.bottom - scale_px(hwnd, 8);
            SelectObject(dc, g_sub_font);
```

```
            DrawTextW(dc, L"Пензенская область, 2026–2028", -1, &subtitle, DT_LEFT
| DT_SINGLELINE | DT_VCENTER);
            EndPaint(hwnd, &paint);
            return 0;
        }
        case WM_DESTROY:
            drop_fonts();
            PostQuitMessage(0);
            return 0;
        default:
            break;
        }
        return DefWindowProcW(hwnd, message, wparam, lparam);
    }
}

int WINAPI wWinMain(HINSTANCE instance, HINSTANCE, PWSTR, int show) {
    g_outcome = calculate();

    WNDCLASSEXW window_class{};
    window_class.cbSize = sizeof(window_class);
    window_class.lpfnWndProc = window_proc;
    window_class.hInstance = instance;
    window_class.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    window_class.hbrBackground = CreateSolidBrush(RGB(244, 246, 243));
    window_class.lpszClassName = L"PenzaCropOpt";
    window_class.hIcon = LoadIconW(instance, MAKEINTRESOURCEW(1));
    window_class.hIconSm = window_class.hIcon;
    RegisterClassExW(&window_class);

    const int dpi = static_cast<int>(GetDpiForSystem());
    const int width = MulDiv(860, dpi, 96);
    const int height = MulDiv(700, dpi, 96);
    const int x = (GetSystemMetrics(SM_CXSCREEN) - width) / 2;
    const int y = (GetSystemMetrics(SM_CYSCREEN) - height) / 2;
    HWND hwnd = CreateWindowExW(0, window_class.lpszClassName, L"Оптимизация
посевных площадей",
        WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN, x, y, width, height, nullptr,
nullptr, instance, nullptr);
    ShowWindow(hwnd, show);
    UpdateWindow(hwnd);
    MSG message{};
    while (GetMessageW(&message, nullptr, 0, 0) > 0) {
        if (!IsDialogMessageW(hwnd, &message)) {
            TranslateMessage(&message);
            DispatchMessageW(&message);
        }
    }
    return static_cast<int>(message.wParam);
}
```

Файл setup.icc

```
#define MyAppName "Оптимизация посевных площадей"
#define MyAppVersion "1.0"
#define MyAppExe "PenzaCropOpt.exe"

[Setup]
AppId={{A4C8E1D2-7B33-4F19-9C55-2E6D8A1B4F70}
AppName={#MyAppName}
AppVersion={#MyAppVersion}
AppPublisher=Индивидуальное задание, тема 3
DefaultDirName={localappdata}\PenzaCropOpt
DefaultGroupName={#MyAppName}
DisableProgramGroupPage=yes
PrivilegesRequired=lowest
OutputDir=..
OutputBaseFilename=Установщик
SetupIconFile=..\src\app.ico
UninstallDisplayIcon={app}\{#MyAppExe}
UninstallDisplayName={#MyAppName}
Compression=lzma2
SolidCompression=yes
WizardStyle=modern
ArchitecturesAllowed=x64compatible
ArchitecturesInstallIn64BitMode=x64compatible
VersionInfoVersion=1.0.0.0
VersionInfoDescription=Установщик программы оптимизации посевных площадей

[Languages]
Name: "russian"; MessagesFile: "compiler:Languages\Russian.isl"

[Files]
Source: "..\bin\x64\Release\{#MyAppExe}"; DestDir: "{app}"; Flags: ignoreversion
Source: "..\bin\x64\Release\data\*"; DestDir: "{app}\data"; Flags: ignoreversion
recursesubdirs

[Icons]
Name: "{group}\{#MyAppName}"; Filename: "{app}\{#MyAppExe}"; WorkingDir: "{app}"
Name: "{autodesktop}\{#MyAppName}"; Filename: "{app}\{#MyAppExe}"; WorkingDir:
"{app}"

[Run]
Filename: "{app}\{#MyAppExe}"; Description: "Запустить программу"; Flags: nowait
postinstall skipifsilent
```